

3	5.75	19.71
4	5.43	22.14
5	4.99	27.13
6	2.52	29.65
7	4.61	34.26
8	3.91	38.17
Database Schema		
social_media		
name		type
username		TEXT
month		INTEGER
change_in_followers		REAL
change_in_following		INTEGER
posts		INTEGER
Rows: 64		

Query Results				
month	change_in_followers	running_total	running_avg	running_count
1	6.27	6.27	6.27	1
2	4.71	10.98	5.49	2
3	5.73	16.71	5.57	3
4	5.43	22.14	5.535	4
5	4.99	27.13	5.426	5
6	2.52	29.65	4.94166666666667	6
7	4.61	34.26	4.89428571428572	7
8	3.91	38.17	4.77125	8
Database Schema				
social_media				
name			type	
username			TEXT	
month			INTEGER	
change_in_followers			REAL	
change_in_following			INTEGER	
posts			INTEGER	
Rows: 64				

Query Results	
username	total_follower_change
aliaabhatt	8.2
arianagrande	34.35
codecademy	0.17
cristiano	42.44
instagram	38.17
sonnynomnom	0.0
theellenshow	11.42
therock	29.68
Database Schema	
social_media	
name	type
username	TEXT
month	INTEGER
change_in_followers	REAL
change_in_following	INTEGER
posts	INTEGER
Rows: 64	

Query Results		
username	month	total_follower_change
aliaabhatt	1	1.15
aliaabhatt	2	1.16
aliaabhatt	3	1.52
aliaabhatt	4	1.57
aliaabhatt	5	1.47
aliaabhatt	6	-0.76
aliaabhatt	7	1.09
aliaabhatt	8	1.0
arianagrande	1	3.69
arianagrande	2	4.4
arianagrande	3	4.93
arianagrande	4	4.12
arianagrande	5	5.47
arianagrande	6	4.01
arianagrande	7	3.71
arianagrande	8	4.02
codecademy	1	0.03
codecademy	2	0.01
codecademy	3	0.02
codecademy	4	0.01
codecademy	5	0.01
codecademy	6	0.01
codecademy	7	0.03

Query Results			
username	month	change_in_followers	running_total_followers_change
aliaabhatt	1	1.15	1.15
aliaabhatt	2	1.16	2.31
aliaabhatt	3	1.52	3.83
aliaabhatt	4	1.57	5.4
aliaabhatt	5	1.47	6.87
aliaabhatt	6	-0.76	6.11
aliaabhatt	7	1.09	7.2
aliaabhatt	8	1.0	8.2
arianagrande	1	3.69	3.69
arianagrande	2	4.4	8.09
arianagrande	3	4.93	13.02
arianagrande	4	4.12	17.14
arianagrande	5	5.47	22.61
arianagrande	6	4.01	26.62
arianagrande	7	3.71	30.33
arianagrande	8	4.02	34.35
codecademy	1	0.03	0.03

Query Results					
username	month	change_in_followers	running_total_followers_change	posts	running
aliaabhatt	1	1.15	1.15	9	
aliaabhatt	2	1.16	2.31	14	
aliaabhatt	3	1.52	3.83	25	
aliaabhatt	4	1.57	5.4	9	
aliaabhatt	5	1.47	6.87	9	
aliaabhatt	6	-0.76	6.11	7	
aliaabhatt	7	1.09	7.2	5	
aliaabhatt	8	1.0	8.2	13	
arianagrande	1	3.69	3.69	12	
arianagrande	2	4.4	8.09	22	
arianagrande	3	4.93	13.02	14	

Query Results		
month	change_in_followers	running_total
1	6.27	6.27
2	4.71	10.98
3	5.73	16.71
4	5.43	22.14
5	4.99	27.13
6	2.52	29.65
7	4.61	34.26
8	3.91	38.17

Database Schema	
social_media	
name	type
username	TEXT
month	INTEGER
change_in_followers	REAL
change_in_following	INTEGER
posts	INTEGER

Rows: 64

Query Results		
username	posts	fewest_posts
aliaabhattacharya	5	5
aliaabhattacharya	7	5

aliaabhata	9	5
aliaabhata	9	5
aliaabhata	9	5
aliaabhata	13	5
aliaabhata	14	5
aliaabhata	25	5
arianagrande	12	12
arianagrande	14	12
arianagrande	14	12

Query Results		
username	posts	fewest_posts
aliaabhata	5	5
aliaabhata	7	7

aliaabhatt	9	9
aliaabhatt	9	9
aliaabhatt	9	9
aliaabhatt	13	13
aliaabhatt	14	14
aliaabhatt	25	25
arianagrande	12	12
arianagrande	14	14
arianagrande	14	14
arianagrande	22	22
arianagrande	22	22

Query Results		
username	posts	most_posts
aliaabhatt	5	25
aliaabhatt	7	25

aliaabhatt	9	25
aliaabhatt	9	25
aliaabhatt	9	25
aliaabhatt	13	25
aliaabhatt	14	25
aliaabhatt	25	25
arianagrande	12	48
arianagrande	14	48
arianagrande	14	48
arianagrande	22	48
arianagrande	22	48

Query Results			
artist	week	streams_millions	chart_position
Drake	1	288.2	1
Drake	2	160.9	1

Drake	3	140.0	2
Drake	4	131.8	2
Drake	5	123.7	1
Drake	6	126.2	1
Drake	7	122.6	1
Drake	8	117.6	1
The Weeknd	1	76.3	6
The Weeknd	2	72.6	9
The Weeknd	3	69.9	9

Query Results			
artist	week	streams_millions	previous_week_streams
Lady Gaga	1	15.4	0
Lady Gaga	2	15.2	15.4

Lady Gaga	3	16.6	15.2
Lady Gaga	4	21.0	16.6
Lady Gaga	5	64.0	21.0
Lady Gaga	6	36.0	64.0
Lady Gaga	7	30.5	36.0
Lady Gaga	8	27.0	30.5
Database Schema			
streams			
name		type	
artist		TEXT	
week		INTEGER	
streams_millions		REAL	
chart_position		INTEGER	
Rows: 64			

Query Results			
artist	week	streams_millions	streams_millions_change
Lady Gaga	1	15.4	0.0
Lady Gaga	2	15.2	-0.2000000000000001

Lady Gaga	3	16.6	1.4
Lady Gaga	4	21.0	4.4
Lady Gaga	5	64.0	43.0
Lady Gaga	6	36.0	-28.0
Lady Gaga	7	30.5	-5.5
Lady Gaga	8	27.0	-3.5
Database Schema			
streams			
name		type	
artist		TEXT	
week		INTEGER	
streams_millions		REAL	
chart_position		INTEGER	
Rows: 64			

Query Results			
artist	week	streams_millions	streams_millions_change
Lady Gaga	1	15.4	0.0
Lady Gaga	2	15.2	-0.2000000000000001

Lady Gaga	5	16.6	1.4
Lady Gaga	4	21.0	4.4
Lady Gaga	5	64.0	43.0
Lady Gaga	6	36.0	-28.0
Lady Gaga	7	30.5	-5.5
Lady Gaga	8	27.0	-3.5
Database Schema			
streams			
name		type	
artist		TEXT	
week		INTEGER	
streams_millions		REAL	
chart_position		INTEGER	
Rows: 64			

Query Results			
artist	week	streams_millions	streams_millions_change
Bad Bunny	1	33.7	35.9
Bad Bunny	2	69.6	-10.6
Bad Bunny	3	59.0	-9.3
Bad Bunny	4	49.7	-7.400000000000001
Bad Bunny	5	42.3	0.5
Bad Bunny	6	42.8	-1.099999999999999
Bad Bunny	7	41.7	-2.400000000000001
Bad Bunny	8	39.3	Ø
Beyoncé	1	26.3	-0.4000000000000002
Beyoncé	2	25.9	0.7000000000000003
Beyoncé	3	26.6	-0.1000000000000001
Beyoncé	4	26.5	-0.199999999999999
Beyoncé	5	26.3	1.6
Beyoncé	6	27.9	-0.599999999999998

Query Results			
artist	week	streams_millions	streams_millions_change

Bad Bunny	1	33.7	35.9
Bad Bunny	2	69.6	-10.6
Bad Bunny	3	59.0	-9.3
Bad Bunny	4	49.7	-7.400000000000001
Bad Bunny	5	42.3	0.5
Bad Bunny	6	42.8	-1.099999999999999
Bad Bunny	7	41.7	-2.400000000000001
Bad Bunny	8	39.3	∅
Beyoncé	1	26.3	-0.4000000000000002
Beyoncé	2	25.9	0.7000000000000003
Beyoncé	3	26.6	-0.1000000000000001
Beyoncé	4	26.5	-0.199999999999999
Beyoncé	5	26.3	1.6

Query Results			
row_num	artist	week	streams_millions
1	Lady Gaga	2	15.2
2	Lady Gaga	1	15.4

3	Lady Gaga	3	16.6
4	Lady Gaga	4	21.0
5	Doja Cat	8	24.8
6	Beyoncé	2	25.9
7	Beyoncé	1	26.3
8	Beyoncé	5	26.3
9	Doja Cat	7	26.5
10	Beyoncé	4	26.5

Query Results			
row_num	artist	week	streams_millions
1	Lady Gaga	2	15.2
2	Lady Gaga	1	15.4
3	Lady Gaga	3	16.6
4	Lady Gaga	4	21.0
5	Doja Cat	8	24.8
6	Beyoncé	2	25.9
7	Beyoncé	1	26.3
8	Beyoncé	5	26.3
9	Doja Cat	7	26.5
10	Beyoncé	4	26.5
11	Beyoncé	3	26.6
12	Lady Gaga	8	27.0
13	Beyoncé	7	27.3

Query Results			
row_num	artist	week	streams_millions
1	Drake	1	288.2

2	Drake	2	160.9
3	Drake	3	140.0
4	Drake	4	131.8
5	Drake	6	126.2
6	Drake	5	123.7
7	Drake	7	122.6
8	Drake	8	117.6
9	The Weeknd	1	76.3
10	The Weeknd	2	72.6
11	The Weeknd	4	71.4
12	The Weeknd	3	69.9
13	Bad Bunny	2	69.6

Query Results			
rank	artist	week	streams_millions
1	Lady Gaga	2	15.2
2	Lady Gaga	1	15.4

3	Lady Gaga	3	16.6
4	Lady Gaga	4	21.0
5	Doja Cat	8	24.8
6	Beyoncé	2	25.9
7	Beyoncé	1	26.3
7	Beyoncé	5	26.3
9	Doja Cat	7	26.5
9	Beyoncé	4	26.5

Query Results			
rank	artist	week	streams_millions
1	Drake	1	288.2
2	Drake	2	160.9
3	Drake	3	140.0
4	Drake	4	131.8
5	Drake	6	126.2
6	Drake	5	123.7
7	Drake	7	122.6
8	Drake	8	117.6
9	The Weeknd	1	76.3
10	The Weeknd	2	72.6
11	The Weeknd	4	71.4

Query Results			
rank	artist	week	streams_millions
1	Drake	1	288.2
2	The Weeknd	1	76.3
3	Luke Combs	1	55.8
4	Taylor Swift	1	47.7
5	Doja Cat	1	41.7
6	Bad Bunny	1	33.7
7	Beyoncé	1	26.3
8	Lady Gaga	1	15.4
1	Drake	2	160.9
2	The Weeknd	2	72.6
3	Bad Bunny	2	69.6
4	Luke Combs	2	52.1

Query Results			
---------------	--	--	--

quartile	artist	week	streams_millions
1	Drake	1	288.2
1	The Weeknd	1	76.3
2	Luke Combs	1	55.8
2	Taylor Swift	1	47.7
3	Doja Cat	1	41.7
3	Bad Bunny	1	33.7
4	Beyoncé	1	26.3
4	Lady Gaga	1	15.4
1	Drake	2	160.9
1	The Weeknd	2	72.6
2	Bad Bunny	2	69.6

True/False: **LAG** compares with the previous row while **LEAD** compares with the next row.

False

True

Employee	Hire Date	Salary	LAG(Salary)	LEAD(S:
Alice	2020-01-01	\$50k	NULL	\$55k
Bob	2020-06-01	\$55k	\$50k	\$60k
Charlie	2021-01-01	\$60k	\$55k	NULL

Below we have a query for our artist streaming data. Fill in the blanks so that our window function will return the **quartiles** of our **weekly_streams**.

```
SELECT
```

```
OVER (
```

```
ORDER BY streams_millions DESC
) weekly_streams_quantile,
artist,
week,
streams_millions
FROM
artist_streams;
```

NTILE(4)

NTILE() * 4

4TILE()

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Finish the query below so that it is showing you a running total of the `duration_seconds` for a bike race ordered by the racer's `start_time`.

```
SELECT
  duration_seconds,
  start_time,
```

```
    ) AS running_total  
FROM  
    bike_race_data;
```

OVER

ORDER BY

GROUP BY

SUM(start_time)

SUM(duration_seconds)

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Finish the query below so that it is showing you a running total of the `duration_seconds` for a bike race ordered by the racer's `start_time`.

```
SELECT  
    duration_seconds,  
    start_time,  
    SUM(duration_seconds) OVER (  
        ORDER BY start_time  
    ) AS running_total  
FROM  
    bike_race_data;
```

GROUP BY

SUM(start_time)

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Row	Racer	Duration (sec)	Running Total Formula	Running Total	
1	Alice	300	300	300	
2	Bob	250	300 + 250	550	
3	Charlie	320	550 + 320	870	
4	Diana	280	870 + 280	1,150	

From our social media dataset showing how many posts each user posted every month, what should the name of the column `how_many_posts` be changed to?

```
SELECT
  username,
  monthly_posts,
  FIRST_VALUE (monthly_posts) OVER (
```

```

FIRST_VALUE (monthly_posts) OVER (
    PARTITION BY username ORDER BY monthly_posts
) how_many_posts
FROM
    social_media;

average_posts

most_posts

total_posts

fewest_posts

```

Username	monthly_posts	Sorted Order (Ascending)	FIRST_VALUE Result	
Alice	50	Row 1 (First)	50	
Alice	120	Row 2	50	
Alice	85	Row 3	50	

True/False: Window functions are different from aggregate functions because they do not cause rows to become grouped into a single result row.

False

True

Feature	Aggregate Functions	Window Functions	
Row Preservation	 Collapses rows into single result	 Preserves all original rows	
Requires GROUP BY	Yes (when grouping)	No	
Uses OVER Clause	No	Yes	
Output Row Count	Fewer than input (grouped)	Same as input	

We want to see how weekly streams have changed from the previous week for Bad Bunny. What is wrong with the query we are running to get that information?

```
SELECT
  artist,
  week,
  streams_millions,
```

```
streams_millions - LEAD(streams_millions, 1, streams_millions) OVER (  
  ORDER BY week  
) streams_millions_change  
FROM  
  artist_streams  
WHERE  
  artist = 'Bad Bunny';
```

We need to use `LAG` instead of `LEAD`

Nothing, this is correct.

We should subtract `streams_in_millions` from our `LEAD` function

Add `PARTITION BY artist`

We want to see how weekly streams have changed from the previous week for Bad Bunny. What is wrong with the query we are running to get that information?

```
SELECT  
  artist,  
  week,  
  streams_millions,  
  streams_millions - LEAD(streams_millions, 1, streams_millions) OVER (  
    ORDER BY week  
  ) streams_millions_change  
FROM  
  artist_streams  
WHERE  
  artist = 'Bad Bunny';
```

We need to use `LAG` instead of `LEAD`



Correct! `LAG` will compare our current row with the previous row

Nothing, this is correct.

We should subtract `streams_in_millions` from our `LEAD` function

Add `PARTITION BY artist`

Function	Direction	What It Returns
LAG()	⬅ Backward (previous row)	The value from the row before row
LEAD()	➡ Forward (next row)	The value from the row after row

Week	Streams (M)	LAG (prev week)	LEAD (next week)	LAG Change (correct)	LEAD Change (wrong)
1	100	NULL	120	NULL	100 – 120 = -20
2	120	100	90	120 – 100 = +20	120 – 90 = +30
3	90	120	110	90 – 120 = -30	90 – 110 = -20
4	110	90	NULL	110 – 90 = +20	NULL

The following code snippet is attempting to display the *average salary for each department* but is incorrect. Which statement will correct the window function to show salary by department?

```
SELECT employee_name,  
       department,  
       AVG(salary) OVER (  
         PARTITION BY employee_name  
       ) AS 'avg_department_salary'  
FROM employees;
```

ORDER BY salary

PARTITION BY salary

PARTITION BY department

AVG(department)

Option	Problem
<code>ORDER BY salary</code>	This sorts rows <i>within</i> a partition but doesn't fix the partition column
<code>PARTITION BY salary</code>	Groups employees with the same salary together- department
<code>AVG(department)</code>	Invalid! You can't average text/string fields like <code>department</code>

Employee	Dept	Salary	Avg by <i>Employee</i> (Wrong)	Avg by <i>Department</i>
Alice	Sales	\$50k	\$50k (just herself)	\$55k (Alice & Bob)
Bob	Sales	\$60k	\$60k (just himself)	\$55k (Alice & Bob)
Charlie	Eng	\$70k	\$70k (just himself)	\$75k (Charlie & Diana)
Diana	Eng	\$80k	\$80k (just herself)	\$75k (Charlie & Diana)

Which of the following statements is **false** about **ROW** and **RANK** window functions?

ROW and **RANK** are interchangeable and display results in the same way.

RANK will show equal resultant values as tied.

ROW adds a sequential integer to each row.

Like other window functions, **ROW** and **RANK** use the **OVER** clause.

Function	Behavior with Ties	Example Output	
<code>ROW_NUMBER()</code>	Assigns unique sequential numbers, even for ties	1, 2, 2, 4	
<code>RANK()</code>	Assigns the same rank for ties, then skips numbers	1, 2, 2, 4	
<code>DENSE_RANK()</code>	Assigns the same rank for ties, then does not skip numbers	1, 2, 3, 3	

DENSE_RANK ()	Assigns the same rank for ties, no skipping	1, 2, 2, 3	
----------------	--	------------	--

Product	Sales	ROW_NUMBER()	RANK()	Why Different?	
A	\$100K	1	1	First row, highest sales	
B	\$100K	2	1	Tied with A — ROW_NUMBER forces uniqueness, RANK shares rank	
C	\$90K	3	3	RANK skips to 3 (because two rows had rank 1)	
D	\$80K	4	4	Normal progression	

Statement	Why It's True	
"RANK will show equal resultant values as tied."	✓ RANK() gives identical rank numbers to rows with equal values (ties share the same rank).	
"ROW adds a sequential integer to each row."	✓ ROW_NUMBER() generates unique, sequential integers (1, 2, 3, 4...) for every	

	row, regardless of duplicates.	
"Like other window functions, ROW and RANK use the OVER clause."	✓ All ranking window functions (<code>ROW_NUMBER()</code> , <code>RANK()</code> , <code>DENSE_RANK()</code>) require <code>OVER()</code> to define the partition and ordering.	

Query Results					
id	item_name	price	quantity	discount	order_date
1	Donut	2.0	36	0.20 off	2020-08-16 08:04:23
2	Cookie	2.0	18	0.25 off	2020-08-16 09:43:00
3	Bagel	1.5	28	0.10 off	2020-08-16 11:25:12
4	Donut	2.0	11	0.20 off	2020-08-16 11:45:41
5	Sandwich	6.0	23	0.75 off	2020-08-16 12:18:50
Database Schema					
bakery					
name			type		
id			INTEGER		
item_name			TEXT		
price			REAL		
quantity			INTEGER		
discount			TEXT		
order_date			DATETIME		
Rows: 30					
guesses					
name			type		
id			INTEGER		
first_name			TEXT		
last_name			TEXT		
guess			INTEGER		
Rows: 30					

Query Results			
id	first_name	last_name	guess

1	Cherish	Ballantyne	487
2	Delcine	Cheltnam	1156
3	Darby	Vannuchi	784
4	Eduardo	Yansons	1025
5	Katusha	Stockman	720
Database Schema			
bakery			
name		type	
id		INTEGER	
item_name		TEXT	
price		REAL	
quantity		INTEGER	
discount		TEXT	
order_date		DATETIME	
Rows: 30			
guesses			
name		type	
id		INTEGER	
first_name		TEXT	
last_name		TEXT	
guess		INTEGER	
Rows: 30			

Query Results	
price * quantity	
72.0	
36.0	
42.0	

72.0
22.0
138.0
26.0
64.0
35.0
75.0
31.5
120.0
150.0
1.5
150.0

Query Results	
first_name	ABS(guess - 804)
Cherish	317
Delcine	352
Darby	20
Eduardo	221
Katusha	84
Florella	223
Christabel	355
Katheryn	171
Dulcia	211
Aron	28
Jerrilyn	376

Query Results	
ABS(AVG(guess) - 804)	
19.5333333333333	
Database Schema	
guesses	
name	type
id	INTEGER
first_name	TEXT
last_name	TEXT
guess	INTEGER
Rows: 30	

Query Results	
item_name	(price - CAST(discount AS REAL)) * quantity
Donut	64.8
Cookie	31.5
Bagel	39.2
Donut	19.8
Sandwich	120.75
Donut	23.4
Cookie	56.0
Coffee	28.0
Coffee	60.0
Bagel	29.4

Query Results	
DATE(order_date)	
2020-08-16	
2020-08-16	
2020-08-16	

2020-08-16
2020-08-16
2020-08-16
2020-08-16
2020-08-16
2020-08-16
2020-08-16
2020-08-17
2020-08-17
2020-08-17
2020-08-17
2020-08-17
2020-08-17
2020-08-18

Query Results
DATETIME(order_date, 'start of day', '+2 days', '+7 hours')
2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-18 07:00:00

2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-18 07:00:00
2020-08-19 07:00:00
2020-08-19 07:00:00
2020-08-19 07:00:00

Database Schema	
bakery	
name	type
id	INTEGER
item_name	TEXT
price	REAL
quantity	INTEGER
discount	TEXT
order_date	DATETIME
Rows: 30	

Query Results	
order_day	COUNT(*)
16	9
20	6
18	6

17	5
21	2
19	2
Database Schema	
bakery	
name	type
id	INTEGER
item_name	TEXT
price	REAL
quantity	INTEGER
discount	TEXT
order_date	DATETIME
Rows: 30	

Query Results			
state	year	tempf	tempc
Alabama	1895	61.64166667	16.46759259
Alabama	1896	64.26666667	17.92592593
Alabama	1897	64.19166667	17.88425926

Alabama	1898	62.98333333	17.21296296
Alabama	1899	63.1	17.27777778
Alabama	1900	63.40833333	17.44907407
Alabama	1901	61.39166667	16.3287037
Alabama	1902	63.58333333	17.5462963
Alabama	1903	61.975	16.65277778
Alabama	1904	62.76666667	17.09259259
Alabama	1905	62.63333333	17.01851852
Alabama	1906	63.275	17.375
Alabama	1907	63.725	17.625
Alabama	1908	64.04166667	17.80092593

Query Results			
state	year	tempf	running_avg_temp
Alabama	1895	61.64166667	61.64166667
Alabama	1896	64.26666667	62.95416667
Alabama	1897	64.19166667	63.36666667

Alabama	1898	62.98333333	63.270833335
Alabama	1899	63.1	63.236666668
Alabama	1900	63.40833333	63.2652777783333
Alabama	1901	61.39166667	62.9976190485714
Alabama	1902	63.58333333	63.07083333375
Alabama	1903	61.975	62.9490740744444
Alabama	1904	62.76666667	62.930833334
Alabama	1905	62.63333333	62.9037878790909
Alabama	1906	63.275	62.934722225
Alabama	1907	63.725	62.9955128207692

Component	Purpose
AVG(tempf)	Calculates the average temperature value
OVER()	Makes it a window function instead of collapsing r
PARTITION BY state	Creates separate running averages for each state
ORDER BY year	Ensures the average accumulates chronologically

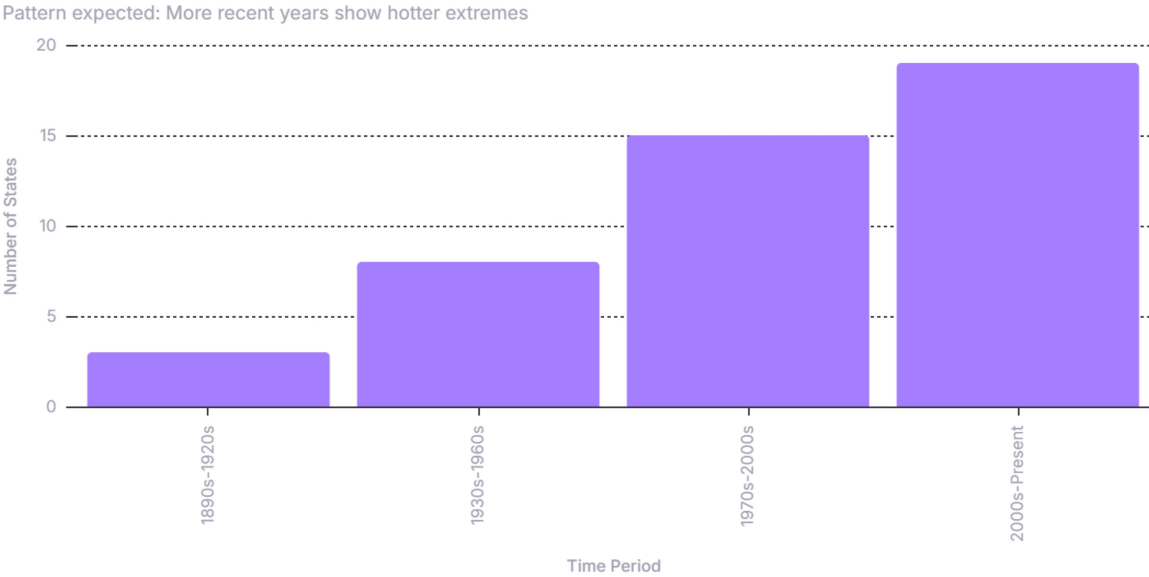
Year	tempf (Actual)	Running Avg (Cumulative)	How Calculate
1895	61.64	61.64	61.64 / 1
1896	64.27	62.96	(61.64 + 64.27)
1897	64.19	63.37	(61.64 + 64.27 ·
1898	62.98	63.27	(sum of 4 years
...	...	...	...

Query Results			
state	year	tempf	lowest_temp
Alabama	1976	60.675	60.675
Alabama	1968	61.0	60.675
Alabama	1940	61.175	60.675
Alabama	1983	61.19166667	60.675
Alabama	1958	61.21666667	60.675
Alabama	1979	61.35833333	60.675
Alabama	1969	61.36666667	60.675
Alabama	1901	61.39166667	60.675
Alabama	1960	61.54166667	60.675
Alabama	1895	61.64166667	60.675
Alabama	1966	61.64166667	60.675
Alabama	1963	61.725	60.675
Alabama	1917	61.85	60.675
Alabama	1978	61.86666667	60.675
Alabama	1967	61.91666667	60.675
Alabama	1961	61.93333333	60.675

Component	Purpose	
MIN(tempf)	Finds the minimum temperature value	
OVER(PARTITION BY state)	Calculates the minimum per state without collapsing rows	
ORDER BY state, tempf ASC	Shows states grouped together, with coldest temps first	

state	year	tempf	highest_temp
Alabama	1921	65.70833333	65.70833333
Alabama	1927	65.58333333	65.70833333
Alabama	2019	65.375	65.70833333
Alabama	2016	65.34166667	65.70833333
Alabama	1911	65.325	65.70833333
Alabama	1922	65.16666667	65.70833333
Alabama	1998	65.125	65.70833333
Alabama	1933	65.1	65.70833333
Alabama	2017	65.03333333	65.70833333
Alabama	1925	64.95833333	65.70833333
Alabama	1990	64.91666667	65.70833333
Alabama	2012	64.91666667	65.70833333
Alabama	2015	64.61666667	65.70833333
Alabama	1938	64.54166667	65.70833333

Component	Purpose
MAX(tempf)	Finds the maximum temperature value
OVER(PARTITION BY state)	Calculates the maximum per state without collapse
ORDER BY state, tempf DESC	Shows states grouped together, with hottest temp



Extreme Type	Expected Pattern	Climate Signal	
Coldest temps	Earlier years (1890s-1930s)	Natural variability, pre-warming baseline	
Hottest temps	Later years (1980s-present)	Global warming signal, increased heat extremes	

Query Results			
state	year	tempf	change_in_temp
MA	1850	40.777777	5.000000

North Dakota	1932	40.63333333	3.01666666
Minnesota	2015	43.49166667	4.85
Montana	1900	42.85	4.75
Michigan	1998	48.25	4.71666667
Wisconsin	1998	47.3	4.65833333
Minnesota	1973	42.26666667	4.6
Wisconsin	2015	44.81666667	4.59166667
Indiana	1921	54.93333333	4.55833333
South Dakota	2012	49.26666667	4.49166667
Indiana	1998	54.95	4.45
Minnesota	1918	40.11666667	4.44166667
Idaho	1934	46.91666667	4.40833334
Idaho	1986	44.075	4.40833333
North Dakota	2015	43.375	4.4
South Dakota	1900	46.16666667	4.39166667
North Dakota	1900	40.61666667	4.36666667
Wisconsin	1973	44.31666667	4.36666666999999
Minnesota	1998	44.90833333	4.34166666
Michigan	1921	46.68333333	4.29166666
Illinois	1998	55.21666667	4.25
Ohio	1921	53.475	4.25
Nevada	1934	52.95833333	4.23333333
Wisconsin	1918	42.18333333	4.20833333
North Dakota	1973	41.76666667	4.2
Illinois	1921	55.0	4.18333333
North Dakota	1997	40.69166667	4.175
Kentucky	1921	58.7	4.166666670000001
Iowa	1918	48.125	4.16666667
Ohio	1998	54.09166667	4.150000000000001

Component	Purpose
<code>LAG(tempf)</code>	Retrieves the previous row's temperature (previous state)
<code>PARTITION BY state</code>	Ensures comparisons only happen within each state timeline
<code>ORDER BY year</code>	Sorts chronologically so "previous" = previous year
Subtraction	<code>tempf - LAG(tempf)</code> = current year minus previous year
<code>IS NOT NULL</code> filter	Removes the first year of each state (no prior data)

State	Year	tempf	LAG (prev year)	change_in_temp	
Alabama	1895	61.64	NULL	<i>(excluded)</i>	
Alabama	1896	64.27	61.64	+2.63	
Alabama	1897	64.19	64.27	-0.08	
Alabama	1898	62.98	64.19	-1.21	
Alabama	1899	63.10	62.98	+0.12	

Region	States	Reason for Large Swings	
Northern Plains / Upper Midwest	ND, SD, MT, MN, NE, IA	Continental climate, far from ocean moderating effects, exposed to Arctic air masses	
Rocky Mountain West	WY, CO, UT	High elevation + continental position = amplified variability	
Southern/Central Plains	KS, OK, TX	Collision of warm Gulf air with cold Arctic air creates volatile swings	

Parameter	Meaning	Your Value
Parameter 1: tempf	Column to look back at	tempf
Parameter 2: 1	How many rows back	1 (previous row)
Parameter 3: tempf	Default value if no prior row exists	tempf (current row's val

Year	tempf	LAG Without Default	LAG With Default	Change Without Filter	Cha Defa
1895	61.64	NULL ❌	61.64 ✅	NULL (excluded)	0.0 (
1896	64.27	61.64	61.64	2.63	2.63
1897	64.19	64.27	64.27	-0.08	-0.0i

Mistake	Problem	Fix
Using <code>LAG(tempf, 1)</code> only	Returns NULL on first row of each state	Add third parameter: <code>LAG(tempf, 1, tempf)</code>
Filtering <code>WHERE change_in_temp IS NOT NULL</code>	Removes rows you shouldn't exclude	Remove the WHERE clause
Missing <code>ORDER BY change_in_temp</code> at end	Results aren't sorted by change	Add final <code>ORDER BY</code> clause

Requirement	Status
☒ All three <code>LAG()</code> parameters used	Yes (<code>LAG(tempf, 1, tempf)</code>)
☒ <code>PARTITION BY state</code>	Yes
☒ <code>ORDER BY year</code>	Yes
☒ Final <code>ORDER BY change_in_temp</code>	Yes
☒ No NULL values in results	Yes (first rows show calculated values)

State	Year	Change (°F)	Notes	
North Dakota	1952	+5.02	🔥 Largest increase	
Minnesota	2015	+4.85	Recent warm year	
Montana	1900	+4.75	Early 20th century	
Michigan	1998	+4.72	Late 20th century	
Wisconsin	1998	+4.65	Late 20th century	

Factor	Why It Causes Large Swings
Distance from ocean	No maritime moderation → extremes allowed

Arctic air exposure	Direct path for cold Canadian/Arctic air masses
Continental climate	Greater seasonal variation, higher volatility
Storm track corridor	Frequent weather system transitions

Query Results			
state	year	tempf	coldest_rank
North Dakota	1950	34.9	1
North Dakota	1951	35.61666667	2
Minnesota	1917	35.675	3

North Dakota	1916	35.73333333	4
North Dakota	1917	35.91666667	5
North Dakota	1899	36.25	6
North Dakota	1896	36.425	7
Minnesota	1950	36.45833333	8
Maine	1904	36.51666667	9
North Dakota	1996	36.51666667	9
North Dakota	1907	36.63333333	11
North Dakota	1904	36.93333333	12
Maine	1917	36.94166667	13
Maine	1926	37.13333333	14
Minnesota	1916	37.15833333	15
Montana	1916	37.15833333	15
Minnesota	1951	37.16666667	17
North Dakota	1895	37.175	18
North Dakota	1979	37.225	19
Minnesota	1904	37.3	20
North Dakota	1929	37.3	20
North Dakota	1897	37.425	22
Maine	1907	37.43333333	23
Montana	1951	37.43333333	23
North Dakota	1903	37.46666667	25
Minnesota	1907	37.49166667	26
North Dakota	1927	37.5	27
Minnesota	1929	37.54166667	28
North Dakota	1972	37.56666667	29

Component	Purpose
<code>RANK()</code>	Assigns rank numbers to rows based on ordering
<code>OVER (ORDER BY tempf ASC)</code>	Orders by temperature ascending (coldest first)
<code>No PARTITION BY</code>	Entire dataset is one big window — global ranking states
<code>Final ORDER BY coldest_rank</code>	Displays results sorted by rank (coldest on top)

Reason	Why It Supports Historic Cold Extremes	
Global warming trend	Average temperatures have risen since mid-20th century, making new record lows less likely	
Natural climate variability	Early 20th century had more extreme weather fluctuations (Little Ice Age aftermath, volcanic activity)	
Less urban heat island effect	Older stations were often in less developed areas with fewer heat-retaining surfaces	
Climate change signal	Recent decades show fewer extreme cold events, more frequent record highs instead	

Mistake	Problem	Fix	
Adding <code>PARTITION BY state</code>	Each state starts ranking at 1 → not truly the coldest overall	Remove <code>PARTITION BY</code> for global ranking	
Using <code>ORDER BY tempf DESC</code>	Gets hottest temperatures instead of coldest	Use <code>ASC</code> for coldest-first	
Forgetting final <code>ORDER BY</code>	Results aren't displayed by rank	Add <code>ORDER BY coldest_rank</code> at the end	

State	Appearances in Top 29	Notable
North Dakota	13+ times	Dominates coldest records
Minnesota	6 times	Second most frequent
Maine	5 times	Eastern representation
Montana	3 times	Western presence

Time Period	Count in Top 29	Percentage
1890s-1920s (Early)	17	~59%
1950s-1990s (Later)	7	~24%
Undetermined	~5	~17%

Factor	Why It Creates Historic Cold Extremes
Global warming trend	Post-1950s average temps rising → fewer new re
North Dakota's geography	Far from ocean moderating effects, direct Arctic a
Early measurement era	Less urban heat island effect from development
Natural variability	Early 20th century had more extreme weather fluc



state	year	tempf	warmest_rank
Alabama	1921	65.70833333	1
Alabama	1927	65.58333333	2
Alabama	2019	65.375	3
Alabama	2016	65.34166667	4
Alabama	1911	65.325	5
Alabama	1922	65.16666667	6
Alabama	1998	65.125	7
Alabama	1933	65.1	8
Alabama	2017	65.03333333	9
Alabama	1925	64.95833333	10
Alabama	1990	64.91666667	11
Alabama	2012	64.91666667	11
Alabama	2015	64.61666667	13
Alabama	1938	64.54166667	14
Alabama	2018	64.45	15
Alabama	2007	64.4	16
Alabama	1896	64.26666667	17
Alabama	1949	64.225	18
Alabama	2006	64.2	19

Component	Purpose
RANK()	Assigns rank numbers to rows based on temperat
PARTITION BY state	Each state gets its own independent ranking (ranl

	for that state)
ORDER BY tempf DESC	Descending order puts highest temperatures first rank 1)
Final ORDER BY state, warmest_rank	Groups results by state, sorted by rank within eac

State	Year	tempf	warmest_rank	
Alabama	1921	67.34	1	
Alabama	1925	66.87	2	
Alabama	1998	66.52	3	
Alabama	1900	65.91	4	
Alabama	2017	65.83	5	

Comparison	Coldest Records	Warmest Records
Time clustering	Earlier years (1890s-1930s)	Later years (1980s-present)
Geographic dominance	Northern Plains (ND, MN, MT)	Spread across South/Southwest
Climate signal	Pre-warming baseline	Warming acceleration

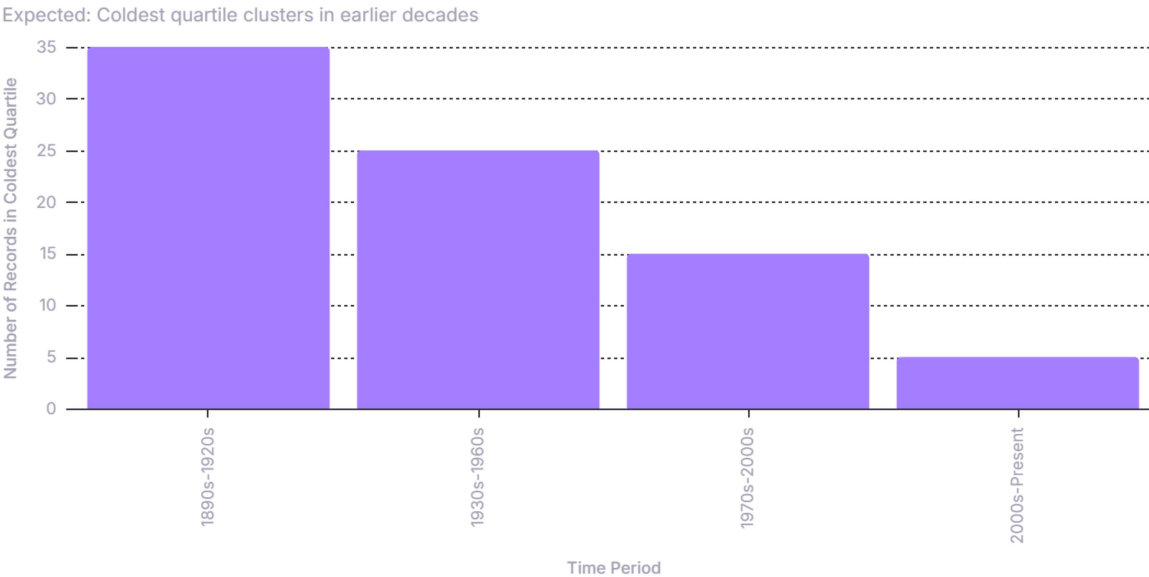
Mistake	Problem	Fix
Using <code>ASC</code> instead of <code>DESC</code>	Gets coldest temps per state, not warmest	Use <code>ORDER BY tempf DE</code>
Forgetting <code>PARTITION BY state</code>	Returns global warmest, not per-state	Add <code>PARTITION BY stat</code>
Using <code>ROW_NUMBER()</code>	Breaks ties arbitrarily	<code>RANK()</code> preserves ties — the same temp share the s

quartile	year	state	tempf
1	1895	Alabama	61.64166667
1	1901	Alabama	61.39166667
1	1903	Alabama	61.975
1	1912	Alabama	62.11666667
1	1917	Alabama	61.85
1	1920	Alabama	62.3
1	1924	Alabama	62.25
1	1940	Alabama	61.175
1	1958	Alabama	61.21666667
1	1960	Alabama	61.54166667
1	1961	Alabama	61.93333333
1	1963	Alabama	61.725
1	1964	Alabama	62.39166667
1	1966	Alabama	61.64166667
1	1967	Alabama	61.91666667
1	1968	Alabama	61.0
1	1969	Alabama	61.36666667
1	1970	Alabama	62.11666667
1	1976	Alabama	60.675

Component	Purpose
NTILE(4)	Divides each state's years into 4 equal-sized groups
PARTITION BY state	Each state gets its own independent quartile assignment

ORDER BY tempf ASC	Sorts from coldest to warmest, so quartile 1 = co
Final ORDER BY state, quartile, year	Groups results by state, then by quartile, then chr

Quartile	Temperature Range	Number of Years	Description	
1	Coldest 25%	~25 years	Lowest temps	
2	25th-50th percentile	~25 years	Below-average temps	
3	50th-75th percentile	~25 years	Above-average temps	
4	Warmest 25%	~25 years	Highest temps	



Metric	Time Distribution	Climate Signal
Coldest individual records (global ranking)	1890s-1930s (historic)	Pre-warming baseline
Coldest per-state records (rank 1)	Mostly historic	Warming made new record
Coldest quartile (quartile 1)	Should skew earlier	Same pattern as above
Warmest records (descending rank)	1980s-present (recent)	Warming acceleration
Largest yearly swings (LAG change)	Northern Plains	Continental climate volatilit

Mistake	Problem	Fix
Using <code>DESC</code> instead of <code>ASC</code>	Quartile 1 = warmest, not coldest	Use <code>ORDER BY tempf AS</code>
Forgetting <code>PARTITION BY state</code>	Global quartiles instead of per-state	Add <code>PARTITION BY stat</code>
Misunderstanding NTILE	NTILE splits by row count , not temperature value	Remember: equal rows, nc

Function	Output	Best Used For	
<code>RANK()</code>	Position (1, 2, 3...)	Finding absolute best/worst	
<code>NTILE(4)</code>	Group (1, 2, 3, 4)	Categorizing into	

		percentiles/quartiles	
LAG()	Previous row value	Year-over-year changes	
SUM() OVER()	Running total	Cumulative sums	
AVG() OVER()	Average per partition	Contextual comparison	

quintile	year	state	tempf
1	1895	North Dakota	37.175
1	1895	Wyoming	37.85833333
1	1895	Minnesota	38.60833333
1	1895	Montana	38.99166667
1	1895	Maine	39.36666667
1	1895	Vermont	40.56666667
1	1895	Wisconsin	41.28333333
1	1895	Idaho	41.40833333
1	1895	New Hampshire	41.53333333
1	1895	Michigan	41.71666667
1	1895	South Dakota	42.41666667
1	1895	Colorado	42.43333333
1	1895	New York	43.45833333
1	1896	North Dakota	36.425
1	1896	Maine	38.33333333
1	1896	Minnesota	38.725
1	1896	Montana	39.43333333
1	1896	Wyoming	39.81666667
1	1896	Vermont	40.725
1	1896	New Hampshire	41.6
1	1896	Wisconsin	42.425
1	1896	South Dakota	42.525
1	1896	Idaho	42.775
1	1896	Michigan	43.05833333

Component	Purpose
<code>NTILE(5)</code>	Divides the entire dataset into 5 equal-sized groups
No <code>PARTITION BY</code>	Global ranking — all states and years compete to
<code>ORDER BY tempf ASC</code>	Sorts coldest to warmest, so quintile 1 = coldest
Final <code>ORDER BY quintile, year</code>	Displays results grouped by quintile, then chronological

Component	Purpose
ROWS BETWEEN 4 PRECEDING AND CURRENT ROW	Creates a moving window of exactly 5 years (not from start)
AVG()	Smooths volatility by averaging across the window
PARTITION BY state	Each state maintains independent rolling calculation

Metric	What It Tells You
avg_abs_change	Typical yearly swing magnitude
max_single_change	Worst-case outlier event
median_change	Middle value (resistant to outliers)

Column	Interpretation
<code>total_range</code>	Complete temperature spread (hot-cold)
<code>warmest_year</code> , <code>coldest_year</code>	Temporal clustering of extremes

State	Total Range	Warmest Year	Coldest Year	Pattern
North Dakota	~30°F+	Recent	Historic	Classic v
California	~20°F	Recent	Historic	Moderate
Florida	~15°F	Recent	Historic	Smallest

Metric	Significance
decadal_change	Magnitude of warming/cooling per decade
trend_label	Qualitative assessment of climate shift

Metric	Meaning
cold_extreme_count	How often this state appears in bottom 5% global
hot_extreme_count	How often this state appears in top 5% globally

State	Cold Extremes	Hot Extremes	Primary Chara
North Dakota	High	Low	Cold-dominant
Texas	Low	High	Hot-dominant
California	Very Low	Very High	Extreme heat s

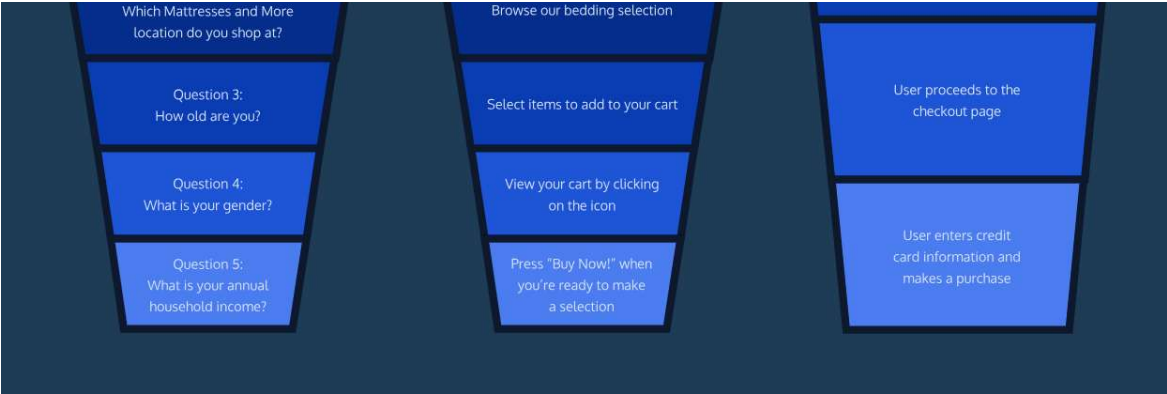
 "Data without insight is noise. Insight without story is forgotten. Story without action is wasted."

Industry	Use Case	Window Function	
E-commerce	Month-over-month revenue growth	LAG(revenue)	
Finance	Stock price running average	AVG(price) OVER (ROWS 30 PRECEDING)	
Healthcare	Patient readmission ranking	RANK() OVER (ORDER BY readmission_rate)	
HR	Salary percentile by department	NTILE(10) OVER (PARTITION BY dept ORDER BY salary)	
Education	Student grade trends over time	LAG(gpa) OVER (PARTITION BY student ORDER BY semester)	
Manufacturing	Defect rate volatility by factory	STDDEV(defects) OVER (PARTITION BY factory)	
Marketing	Customer lifetime value cohort analysis	SUM(ltv) OVER (PARTITION BY signup_month ORDER BY age)	

SQL is not just about querying data—it's about interrogating reality. Every **PARTITION BY** is a decision about fairness. Every **ORDER BY** is a choice about priority.

Every window function is a lens that reveals what was previously invisible.





question_text	num_distinct_users	
1. How likely are you to recommend Mattresses and More...	...	
2. Which Mattresses and More location do you shop at?	...	
3. How likely are you to recommend...	...	

3. How old are you?	...	
4. What is your gender?	...	
5. What is your annual household income?	...	

Query Results	
question_text	num_distinct_users
1. How likely are you to recommend Mattresses and More to a friend?	500
2. Which Mattresses and More location do you shop at?	475
3. How old are you?	380

4. What is your gender?	361
5. What is your annual household income?	270
Database Schema	
survey_responses	
name	type
question_text	TEXT
user_id	TEXT
response	TEXT
Rows: 1986	

Question	Percent Completed this Question
1	100%
2	95%
3	82%
4	95%
5	74%



user_id	modal_text	use
0015585f-51d0-4654-83fc-acce6544b0cf	1 - Welcome to Mattresses and More!	Clo
0028b8b4-abb3-4711-9ea2-0d1ecb975358	1 - Welcome to Mattresses and More!	Clo
0029802c-22a1-4d22-9a59-244341551ec9	1 - Welcome to Mattresses and More!	Clo
0029802c-22a1-4d22-9a59-244341551ec9	2 - Browse our bedding selection	Clo
0029802c-22a1-4d22-9a59-244341551ec9	3 - Select items to add to your cart	Clo
0029802c-22a1-4d22-9a59-244341551ec9	4 - View your cart by clicking on the icon	Clo
0029802c-22a1-4d22-9a59-244341551ec9	5 - Press 'Buy Now!' when you're ready to checkout	Clo
0042ec6f-e343-4486-a009-ecaea4ff31b2	1 - Welcome to Mattresses and More!	Clo
0042ec6f-e343-4486-a009-ecaea4ff31b2	2 - Browse our bedding selection	Clo
0121762e-7d4e-46b0-94d8-7bc56a5cb88b	1 - Welcome to Mattresses and More!	Clo
01baa35a-e111-43f9-8a77-c98f93cbc6be	1 - Welcome to Mattresses and More!	Clo
01dbfec4-a518-4630-9f26-b298d099008f	1 - Welcome to Mattresses and More!	Clo
01dbfec4-a518-4630-9f26-b298d099008f	2 - Browse our bedding selection	Clo
01dbfec4-a518-4630-9f26-b298d099008f	3 - Select items to add to your cart	Clo
01dbfec4-a518-4630-9f26-b298d099008f	4 - View your cart by clicking on the icon	Clo
01dbfec4-a518-4630-9f26-b298d099008f	5 - Press 'Buy Now!' when you're ready to checkout	Clo
023997c1-4261-4770-9fde-94a88c7da6e9	1 - Welcome to Mattresses and More!	Clo
023997c1-4261-4770-9fde-94a88c7da6e9	2 - Browse our bedding selection	Clo
023997c1-4261-4770-9fde-94a88c7da6e9	3 - Select items to add to your cart	Clo
023997c1-4261-4770-9fde-94a88c7da6e9	4 - View your cart by clicking on the icon	Clo

Query Results	
modal_text	num_users
1 - Welcome to Mattresses and More!	1000
2 - Browse our bedding selection	695
3 - Select items to add to your cart	575

4 - View your cart by clicking on the icon	447
5 - Press 'Buy Now!' when you're ready to checkout	379
Database Schema	
onboarding_modals	
name	type
user_id	TEXT
modal_text	TEXT
user_action	TEXT
ab_group	TEXT
Rows: 3096	

Query Results	
modal_text	control_clicks
1 - Welcome to Mattresses and More!	500
2 - Browse our bedding selection	301
3 - Select items to add to your cart	239

4 - View your cart by clicking on the icon	183
5 - Press 'Buy Now!' when you're ready to checkout	152
Database Schema	
onboarding_modals	
name	type
user_id	TEXT
modal_text	TEXT
user_action	TEXT
ab_group	TEXT
Rows: 3096	

modal_text	control_clicks	variant_clicks
1 - Welcome to Mattresses and More!	...	...
2 - Browse our bedding selection	...	...
3 - Select items to add to your cart	...	...
4 - View your cart by clicking on the icon	...	...

5 - Press 'Buy Now!' when you're ready to checkout	...	...
--	-----	-----

Query Results		
modal_text	control_clicks	variant_clicks
1 - Welcome to Mattresses and More!	500	500
2 - Browse our bedding selection	301	394
3 - Select items to add to your cart	239	336
4 - View your cart by clicking on the icon	183	264
5 - Press 'Buy Now!' when you're ready to checkout	152	227
Database Schema		
onboarding_modals		
name	type	
user_id	TEXT	
modal_text	TEXT	
user_action	TEXT	
ab_group	TEXT	
Rows: 3096		

Modal	Control Percent	Variant Percent	
1	100%	100%	
2	60%	79%	
3	80%	85%	
4	80%	80%	
5	85%	85%	

Onboarding Modals

Welcome to Mattresses and More

Browse our bedding selection

Select items to add to your cart

View your cart by clicking
on the icon

Press "Buy Now!" when
you're ready to make
a selection

Query Results		
user_id	browse_date	item_id
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	3
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	22
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	25

336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	24
4596bb1a-7aa9-4ac9-9896-022d871cdcde	2017-12-20	0
user_id	checkout_date	item_id
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24
3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	2017-12-20	7
3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	2017-12-20	6
3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	2017-12-20	12
user_id	purchase_date	item_id
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24
3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	2017-12-20	7
3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	2017-12-20	6
3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	2017-12-20	12
Database Schema		
browse		
name	type	
user_id	TEXT	
browse_date	TEXT	
item_id	INTEGER	
Rows: 2352		
items		
name	type	
id	INTEGER	
item	TEXT	
price	REAL	
Rows: 33		
checkout		

Query Results			
user_id	browse_date	item_id	user_id
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	3	Ø
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	22	Ø
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	25	Ø
336f9fdc-aaeb-48a1-a773-e3a935442d45	2017-12-20	24	Ø
4596bb1a-7aa9-4ac9-9896-022d871cdcde	2017-12-20	0	Ø
4596bb1a-7aa9-4ac9-9896-022d871cdcde	2017-12-20	2	Ø
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	26	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	2017-12-20	24	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e
fc394c75-36f1-4df1-8665-23c32a43591b	2017-12-20	12	Ø
fc394c75-36f1-4df1-8665-23c32a43591b	2017-12-20	24	Ø
fc394c75-36f1-4df1-8665-23c32a43591b	2017-12-20	31	Ø
263e59f2-479b-4736-872c-302ad082b20f	2017-12-20	11	Ø
263e59f2-479b-4736-872c-302ad082b20f	2017-12-20	30	Ø
263e59f2-479b-4736-872c-302ad082b20f	2017-12-20	21	Ø
263e59f2-479b-4736-872c-302ad082b20f	2017-12-20	32	Ø

Query Results			
browse_date	user_id	is_checkout	is_purchase
2017-12-20	336f9fdc-aaeb-48a1-a773-e3a935442d45	0	0
2017-12-20	4596bb1a-7aa9-4ac9-9896-022d871cdcde	0	0
2017-12-20	2fdb3958-ffc9-4b84-a49d-5f9f40e9469e	1	1

2017-12-20	fc394c75-36f1-4df1-8665-23c32a43591b	0	0
2017-12-20	263e59f2-479b-4736-872c-302ad082b20f	0	0
2017-12-20	58ff3291-84bf-4fc7-96cc-0bc1477adea9	0	0
2017-12-20	d582b899-cace-43dc-84f3-a1df0c30e90c	0	0
2017-12-20	3215212f-7a6f-4d95-937a-ee0ce911db04	0	0
2017-12-20	d0768167-da9c-4209-b3e6-5c6fc446bece	0	0
2017-12-20	182fdbb3-babd-4ade-ae6d-c3d6f30ffcde	0	0
2017-12-20	97b8378f-8b66-4f54-9269-0e27c84ab311	0	0
2017-12-20	fb65f3ad-ed9e-4a70-8fa4-ebd609fe7383	0	0
2017-12-20	3a3e5fe6-39a7-4068-8009-3b9f649cb1aa	1	1
2017-12-20	b1bf3547-0c44-4371-907f-31bc75b9c593	1	1
2017-12-20	313624e5-5331-4a05-ad3a-96786c3162a9	1	1
2017-12-20	e2ca4439-a294-4f04-bace-dff782021560	0	0
2017-12-20	2e976863-8aaf-4d6c-8ed9-c25504d72bb1	0	0
2017-12-20	b8073de8-84db-4576-8840-40805497e3b8	0	0
2017-12-20	bb246075-ce55-44df-aedd-32a15be914a6	1	0
2017-12-20	98e28ac7-83ae-4570-bdc8-c5f2a95820a3	0	0
2017-12-20	c7de0c37-4809-4944-bebb-e1ab7a13747b	1	1
2017-12-20	0514100-55-470-00-0015-05-47	0	0

browser_date	user_id	is_checkout	is_purchase	
2017-12-20	6a7617321513	1	0	
2017-12-20	022d871cdcde	0	0	
...	...	...	...	

Query Results
num_browse
775
Database Schema
browse

name		type	
user_id		TEXT	
browse_date		TEXT	
item_id		INTEGER	
Rows: 2352			
items			
name		type	
id		INTEGER	
item		TEXT	
price		REAL	
Rows: 33			
checkout			
name		type	
user_id		TEXT	
checkout_date		TEXT	
item_id		INTEGER	
Rows: 539			
purchase			
name		type	
user_id		TEXT	
purchase_date		TEXT	
item_id		INTEGER	
Rows: 472			

Query Results	
num_browse	num_checkout
775	183
Database Schema	
browse	
name	type
user_id	TEXT
browse_date	TEXT
item_id	INTEGER
Rows: 2352	
items	
name	type
id	INTEGER
item	TEXT
price	REAL
Rows: 33	
checkout	
name	type
user_id	TEXT
checkout_date	TEXT
item_id	INTEGER
Rows: 539	
purchase	
name	type
user_id	TEXT
purchase_date	TEXT
item_id	INTEGER
Rows: 472	

Query Results		
num_browse	num_checkout	num_purchase
775	183	163
Database Schema		
browse		

name		type	
user_id		TEXT	
browse_date		TEXT	
item_id		INTEGER	
Rows: 2352			
items			
name		type	
id		INTEGER	
item		TEXT	
price		REAL	
Rows: 33			
checkout			
name		type	
user_id		TEXT	
checkout_date		TEXT	
item_id		INTEGER	
Rows: 539			
purchase			
name		type	
user_id		TEXT	
purchase_date		TEXT	
item_id		INTEGER	
Rows: 472			

Query Results				
num_browse	num_checkout	num_purchase)	browse_to_checkout	checkout_to_purchase
775	183	163	0.236129032258065	0.890710382513661
Database Schema				
browse				

browse	
name	type
user_id	TEXT
browse_date	TEXT
item_id	INTEGER
Rows: 2352	
items	
name	type
id	INTEGER
item	TEXT
price	REAL
Rows: 33	
checkout	
name	type
user_id	TEXT
checkout_date	TEXT
item_id	INTEGER
Rows: 539	
purchase	
name	type
user_id	TEXT
purchase_date	TEXT
item_id	INTEGER
Rows: 472	

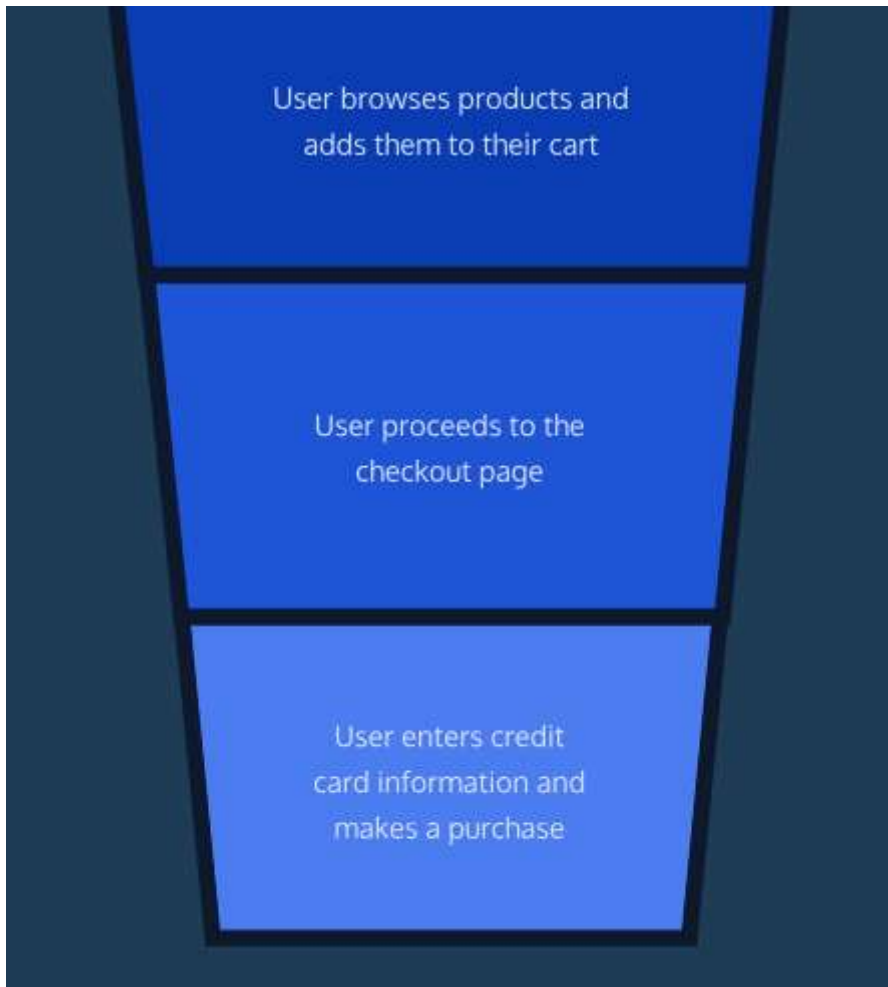
Query Results				
num_browse	num_checkout	num_purchase	browse_to_checkout	checkout_to_purchase
775	183	163	0.236129032258065	0.890710382513661
Database Schema				
browse				

browse	
name	type
user_id	TEXT
browse_date	TEXT
item_id	INTEGER
Rows: 2352	
items	
name	type
id	INTEGER
item	TEXT
price	REAL
Rows: 33	
checkout	
name	type
user_id	TEXT
checkout_date	TEXT
item_id	INTEGER
Rows: 539	
purchase	
name	type
user_id	TEXT
purchase_date	TEXT
item_id	INTEGER
Rows: 472	

browse	checkout	purchase	browse_to_checkout	checkout_to_purchase
775	183	163	0.236	0.890

browse_date	browse	checkout	purchase	browse_to_checkout	checkout_to_purchase	
2017-12-20	100	20	16	0.2	0.8	
2017-12-21	150	33	28	0.22	0.84	
2017-12-22	250	62	55	0.24	0.88	
2017-12-23	275	68	64	0.24	0.94	

Website Usability



Query Results		
question	user_id	response
1. What are you looking for?	005e7f99-d48c-4fce-b605-10506c85aaf7	Women's Style!
2. What's your fit?	005e7f99-d48c-4fce-b605-10506c85aaf7	Medium
3. Which shapes do you like?	00a556ed-f13e-4c67-8704-27e3573684cd	Round
4. Which colors do you like?	00a556ed-f13e-4c67-8704-27e3573684cd	Two-Tone
1. What are you looking for?	00a556ed-f13e-4c67-8704-27e3573684cd	I'm not sure. Let's sl
2. What's your fit?	00a556ed-f13e-4c67-8704-27e3573684cd	Narrow
5. When was your last eye exam?	00a556ed-f13e-4c67-8704-27e3573684cd	<1 Year
3. Which shapes do you like?	00bf9d63-0999-43a3-9e5b-9c372e6890d2	Square
5. When was your last eye exam?	00bf9d63-0999-43a3-9e5b-9c372e6890d2	<1 Year
2. What's your fit?	00bf9d63-0999-43a3-9e5b-9c372e6890d2	Medium
Database Schema		
home_try_on		
name	type	
user_id	TEXT	
number_of_pairs	TEXT	

```
1  ▼ SELECT question,
2      COUNT(DISTINCT user_id)
3  FROM survey
4  GROUP BY question;
```

5

Query Results	
question	COUNT(DISTINCT user_id)
1. What are you looking for?	500
2. What's your fit?	475
3. Which shapes do you like?	380

4. Which colors do you like?	361
5. When was your last eye exam?	270
Database Schema	
home_try_on	
name	type
user_id	TEXT
number_of_pairs	TEXT
address	TEXT
Rows: 750	
purchase	
name	type
user_id	TEXT
product_id	INTEGER
style	TEXT
model_name	TEXT
color	TEXT
price	INTEGER
Rows: 495	
survey	
name	type
question	TEXT
user_id	TEXT
response	TEXT
Rows: 1986	
quiz	
name	type
user_id	TEXT
style	TEXT

$$\frac{\text{cancellations}}{\text{total subscribers}}$$

Query Results	
(100./2000)	
0.05	
Database Schema	
subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

Query Results	
(100./2000)	
0.05	
(450./3000)	
0.15	

Database Schema	
subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

cancellations

total subscribers

Query Results	
result	
0.126353790613718	
Database Schema	
subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

Query Results
1.0 * SUM(is_canceled)/SUM(is_active)
0.126353790613718
Database Schema
subscriptions

subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

Query Results	
first_day	last_day
2017-01-01	2017-01-31
2017-02-01	2017-02-28
2017-03-01	2017-03-31
Database Schema	
subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

Query Results				
id	subscription_start	subscription_end	first_day	last_day
1	2017-03-18	∅	2017-01-01	2017-01-31
1	2017-03-18	∅	2017-02-01	2017-02-28
1	2017-03-18	∅	2017-03-01	2017-03-31

2	2017-03-05	Ø	2017-01-01	2017-01-31
2	2017-03-05	Ø	2017-02-01	2017-02-28
2	2017-03-05	Ø	2017-03-01	2017-03-31
3	2016-12-21	Ø	2017-01-01	2017-01-31
3	2016-12-21	Ø	2017-02-01	2017-02-28
3	2016-12-21	Ø	2017-03-01	2017-03-31
4	2017-03-26	Ø	2017-01-01	2017-01-31
4	2017-03-26	Ø	2017-02-01	2017-02-28
4	2017-03-26	Ø	2017-03-01	2017-03-31
5	2017-03-14	Ø	2017-01-01	2017-01-31
5	2017-03-14	Ø	2017-02-01	2017-02-28
5	2017-03-14	Ø	2017-03-01	2017-03-31
6	2016-12-02	Ø	2017-01-01	2017-01-31
6	2016-12-02	Ø	2017-02-01	2017-02-28
6	2016-12-02	Ø	2017-03-01	2017-03-31

id	subscription_start	subscription_end	month	
1	2016-12-03	2017-02-15	2016-12-01	
1	2016-12-03	2017-02-15	2017-01-01	
1	2016-12-03	2017-02-15	2017-02-01	
1	2016-12-03	2017-02-15	2017-03-01	

month	is_active
-------	-----------

2016-12-01	0
2017-01-01	1
2017-02-01	1
2017-03-01	0

Query Results		
id	month	is_active
1	2017-01-01	0
1	2017-02-01	0
1	2017-03-01	0
2	2017-01-01	0
2	2017-02-01	0
2	2017-03-01	0
3	2017-01-01	1
3	2017-02-01	1
3	2017-03-01	1
4	2017-01-01	0
4	2017-02-01	0
4	2017-03-01	0
5	2017-01-01	0
5	2017-02-01	0
5	2017-03-01	0
6	2017-01-01	1

month	is_active	is_canceled
2016-12-01	0	0
2017-01-01	1	0
2017-02-01	1	1
2017-03-01	0	0

Query Results			
id	month	is_active	is_canceled
1	2017-01-01	0	0
1	2017-02-01	0	0
1	2017-03-01	0	0
2	2017-01-01	0	0
2	2017-02-01	0	0
2	2017-03-01	0	0
3	2017-01-01	1	0
3	2017-02-01	1	0
3	2017-03-01	1	0
4	2017-01-01	0	0
4	2017-02-01	0	0
4	2017-03-01	0	0
5	2017-01-01	0	0
5	2017-02-01	0	0
5	2017-03-01	0	0
6	2017-01-01	1	0
6	2017-02-01	1	0
6	2017-03-01	1	0
7	2017-01-01	0	0
7	2017-02-01	0	0

Query Results		
month	active	canceled
2017-01-01	276	35
2017-02-01	506	63
2017-03-01	667	158
Database Schema		
subscriptions		
name		type
id		INTEGER
subscription_start		TEXT
subscription_end		TEXT
Rows: 1000		

Query Results	
month	churn_rate
2017-01-01	0.126811594202899
2017-02-01	0.124505928853755
2017-03-01	0.23688155922039

Database Schema	
subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

month	churn_rate	
2017-01-01	0.127	
2017-02-01	0.125	
2017-03-01	0.237	

Query Results	
month	churn_rate
2017-01-01	0.126811594202899
2017-02-01	0.124505928853755
2017-03-01	0.23688155922039

Database Schema	
subscriptions	
name	type
id	INTEGER
subscription_start	TEXT
subscription_end	TEXT
Rows: 1000	

Query Results				
page_name	timestamp	user_id	utm_campaign	utm_source
1 - landing_page	2018-01-24 03:12:16	10006	getting-to-know-cool-tshirts	nytimes
2 - shopping_cart	2018-01-24 04:04:16	10006	getting-to-know-cool-tshirts	nytimes
3 - checkout	2018-01-25 23:10:16	10006	weeklynewsletter	email

3 - checkout	2018-01-23 23:10:10	10000	weekly-newsletter	email
1 - landing_page	2018-01-25 20:32:02	10030	ten-crazy-cool-tshirts-facts	buzzfeed
2 - shopping_cart	2018-01-25 23:05:02	10030	ten-crazy-cool-tshirts-facts	buzzfeed
3 - checkout	2018-01-28 13:26:02	10030	retargetting-campaign	email
4 - purchase	2018-01-28 13:38:02	10030	retargetting-campaign	email
1 - landing_page	2018-01-05 18:31:17	10045	getting-to-know-cool-tshirts	nytimes
2 - shopping_cart	2018-01-05 21:16:17	10045	getting-to-know-cool-tshirts	nytimes
3 - checkout	2018-01-09 03:05:17	10045	retargetting-ad	facebook

Database Schema

page_visits

name	type
page_name	TEXT
timestamp	TEXT
user_id	INTEGER
utm_campaign	TEXT
utm_source	TEXT

Rows: 5692

user_id	timestamp	page_name	utm_source
10069	2018-01-02 23:14:01	1 - landing_page	buzzfeed
10069	2018-01-02 23:55:01	2 - shopping_cart	buzzfeed

Query Results				
page_name	timestamp	user_id	utm_campaign	utm_source
1 - landing_page	2018-01-02 23:14:01	10069	ten-crazy-cool-tshirts-facts	buzzfeed
2 - shopping_cart	2018-01-02 23:55:01	10069	ten-crazy-cool-tshirts-facts	buzzfeed
Database Schema				
page_visits				
name		type		
page_name		TEXT		
timestamp		TEXT		
user_id		INTEGER		
utm_campaign		TEXT		
utm_source		TEXT		
Rows: 5692				

user_id	timestamp	page_name	utm_source
10069	2018-01-02 23:14:01	1 - landing_page	buzzfeed
10069	2018-01-02 23:14:01	2 - shopping cart	buzzfeed

10069	2018-01-04 08:12:01	3 - checkout	facebook
10069	2018-01-04 08:13:01	4 - purchase	facebook

Query Results				
page_name	timestamp	user_id	utm_campaign	utm_source
1 - landing_page	2018-01-02 23:14:01	10069	ten-crazy-cool-tshirts-facts	buzzfeed
2 - shopping_cart	2018-01-02 23:55:01	10069	ten-crazy-cool-tshirts-facts	buzzfeed
3 - checkout	2018-01-04 08:12:01	10069	retargetting-ad	facebook

4 - purchase	2018-01-04 08:13:01	10069	retargetting-ad	facebook
Database Schema				
page_visits				
name			type	
page_name			TEXT	
timestamp			TEXT	
user_id			INTEGER	
utm_campaign			TEXT	
utm_source			TEXT	
Rows: 5692				

Query Results				
page_name	timestamp	user_id	utm_campaign	utm_source
1 - landing_page	2018-01-18 05:27:25	10329	interview-with-cool-tshirts-founder	medium
2 - shopping_cart	2018-01-18 07:15:25	10329	interview-with-cool-tshirts-founder	medium
3 - checkout	2018-01-22 16:31:25	10329	retargetting-campaign	email
4 - purchase	2018-01-22 16:35:25	10329	retargetting-campaign	email
Database Schema				
page_visits				
name			type	
page_name			TEXT	
timestamp			TEXT	
user_id			INTEGER	
utm_campaign			TEXT	
utm_source			TEXT	
Rows: 5692				

Query Results	
user_id	last_touch_at
10069	2018-01-04 08:13:01
Database Schema	
page_visits	

name	type
page_name	TEXT
timestamp	TEXT
user_id	INTEGER
utm_campaign	TEXT
utm_source	TEXT
Rows: 5692	

```
SELECT user_id,  
       timestamp,  
       utm_source  
FROM page_visits;
```

```
SELECT user_id,  
       MIN(timestamp) as first_touch_at  
FROM page_visits  
GROUP BY user_id;
```

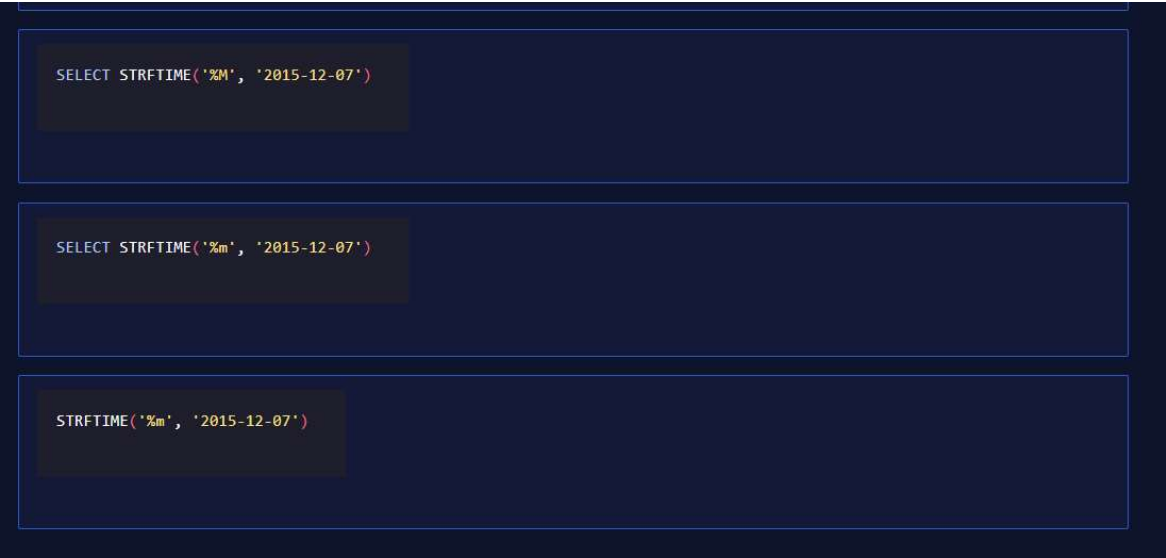
```
page_visits.user_id = first_touch.user_id
```

page_visits.timestamp = first_touch.first_touch_at				
user_id	timestamp	utm_source	first_touch_at	user_id
10048	2018-01-16 04:17:46	medium	2018-01-16 04:17:46	10048
10048	2018-01-16 07:28:46	medium		
10048	2018-01-19 00:00:46	email		
10069	2018-01-02 23:14:01	buzzfeed	2018-01-02 23:14:01	10069
10069	2018-01-02 23:55:01	buzzfeed		
10069	2018-01-04 08:12:01	facebook		
10069	2018-01-04 08:13:01	facebook		
10734	2018-01-05 13:17:16	buzzfeed	2018-01-05 13:17:16	10734

Query Results		
user_id	last_touch_at	utm_source
10069	2018-01-04 08:13:01	facebook
Database Schema		
page_visits		
name		type
page_name		TEXT
timestamp		TEXT
user_id		INTEGER
utm_campaign		TEXT
utm_source		TEXT
Rows: 5692		

You'd like to extract the month from the date `2015-12-07` using SQL. What is the correct function and syntax?

```
SELECT STRFTIME('%Y', '2015-12-07')
```



Option	Code	Result / Problem	
1	STRFTIME('%Y', ...)	Returns '2015' (year)	
2	STRFTIME('%M', ...)	Returns '00' (minutes)	
3	STRFTIME('%m', ...)	Returns '12' (month)	
4	STRFTIME('%m', ...) (no SELECT)	Syntax error – not a full query	

Format	Meaning	Example Result
--------	---------	----------------

%m	Month as decimal (01–12)	12 ← Correct for month
%Y	4-digit Year	2015
%M	Minutes (00–59)	Not a date component
%d	Day of month (01–31)	07
%H	Hour (00–23)	—
%S	Seconds (00–59)	—

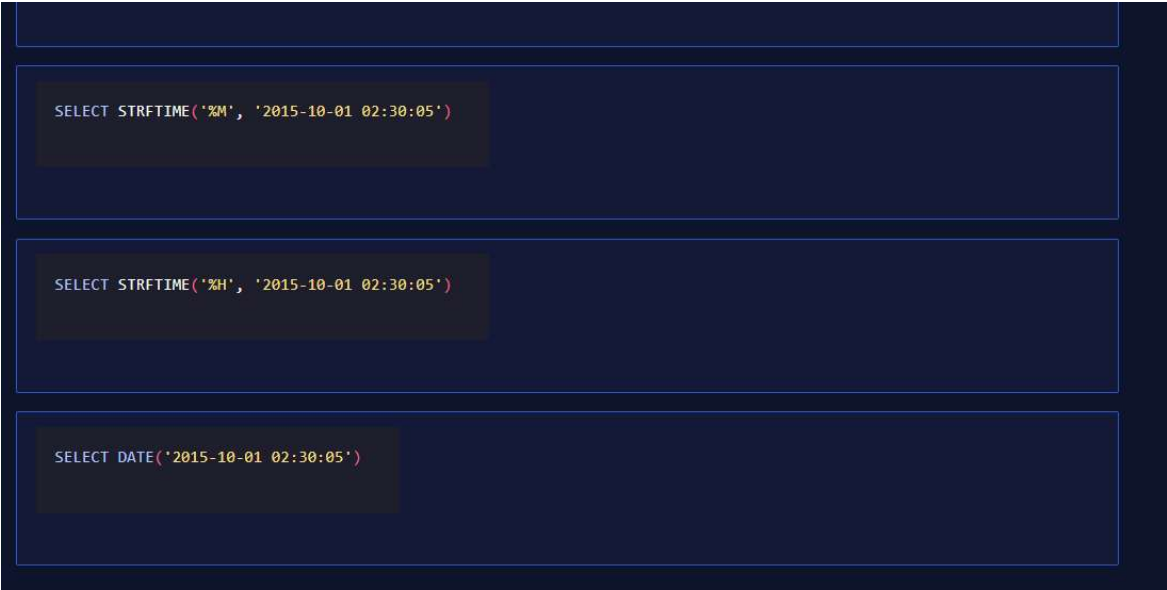
Option	Problem	Result	
STRFTIME('%Y', ...)	Extracts YEAR , not month	Returns 2015	
STRFTIME('%M', ...)	Extracts MINUTES , not month	Returns 00 (no time specified)	
STRFTIME('%m', ...) without SELECT	Invalid SQL syntax (missing keyword)	Syntax Error	

Database	Function	Example
MySQL	MONTH()	SELECT MONTH('2015-12
PostgreSQL	EXTRACT()	SELECT EXTRACT(MONTH 12-07')
SQL Server	DATEPART()	SELECT DATEPART(MONTH-

		<code>07')</code>
SQLite	<code>STRFTIME()</code>	<code>SELECT STRFTIME('%m', 07')</code>

Your input is '2015-10-01 02:30:05'. You worked some magic and got the output '02:30:05'. What was your code?

```
SELECT TIME('2015-10-01 02:30:05')
```



Function	Purpose	Input	Output
<code>TIME()</code>	Extracts the time portion	<code>'2015-10-01 02:30:05'</code>	<code>'02:30:05'</code>
<code>DATE()</code>	Extracts the date portion	<code>'2015-10-01 02:30:05'</code>	<code>'2015-10-01'</code>
<code>STRFTIME()</code>	Formats date/time with specifiers	Varies	Depends on for

Option	What It Actually Does	Result
<code>STRFTIME('%M', ...)</code>	Extracts minutes only	<code>'30'</code>
<code>STRFTIME('%H', ...)</code>	Extracts hours only	<code>'02'</code>
<code>DATE(...)</code>	Extracts the date portion	<code>'2015-10-01'</code>

Specifier	Meaning	Example	
%H	Hour (00–23)	02	
%M	Minute (00–59)	30	
%S	Second (00–59)	05	
%Y	Year (4 digits)	2015	
%m	Month (01–12)	10	
%d	Day (01–31)	01	

You wish to execute the following command:

```
SELECT DATE('2021-07-19', '+1 day');
```

Which of these is a **modifier**?

DATE()

'2021-07-19'

'+1 day'

Component	Role	Example
DATE()	The function itself	N/A
'2021-07-19'	The input date (timestring)	Starting point
'+1 day'	The modifier	Adiusts the date

--	--	--

Modifier	Effect	Example Result
'+1 day'	Add 1 day	2021-07-20
'-3 days'	Subtract 3 days	2021-07-16
'+1 month'	Add 1 month	2021-08-19
'+1 year'	Add 1 year	2022-07-19
'start of month'	First day of current month	2021-07-01
'start of year'	First day of current year	2021-01-01
'weekday 0'	Move to nearest Sunday	Depends on date
'localtime'	Convert to local time zone	Varies
'unixepoch'	Interpret as Unix timestamp	Converts to date

Option	Actual Role	Why Not a Modifier?
<code>DATE()</code>	The function name	It's the operation being per adjustment
<code>'2021-07-19'</code>	The input timestring	It's the data being modified modification itself
<code>'+1 day'</code>	Modifier	 It's the adjustment appl

How would you return the absolute value of the number -3?

SELECT

(

)

ABS

STRFTIME

ABSOLUTE

3

%3

-3

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Query	Result	
SELECT ABS(-3);	3	
SELECT ABS(15);	15	
SELECT ABS(0);	0	
SELECT ABS(-7.5);	7.5	

How would you return the absolute value of the number -3?

SELECT

ABS

(

-3

)

STRFTIME

ABSOLUTE

3

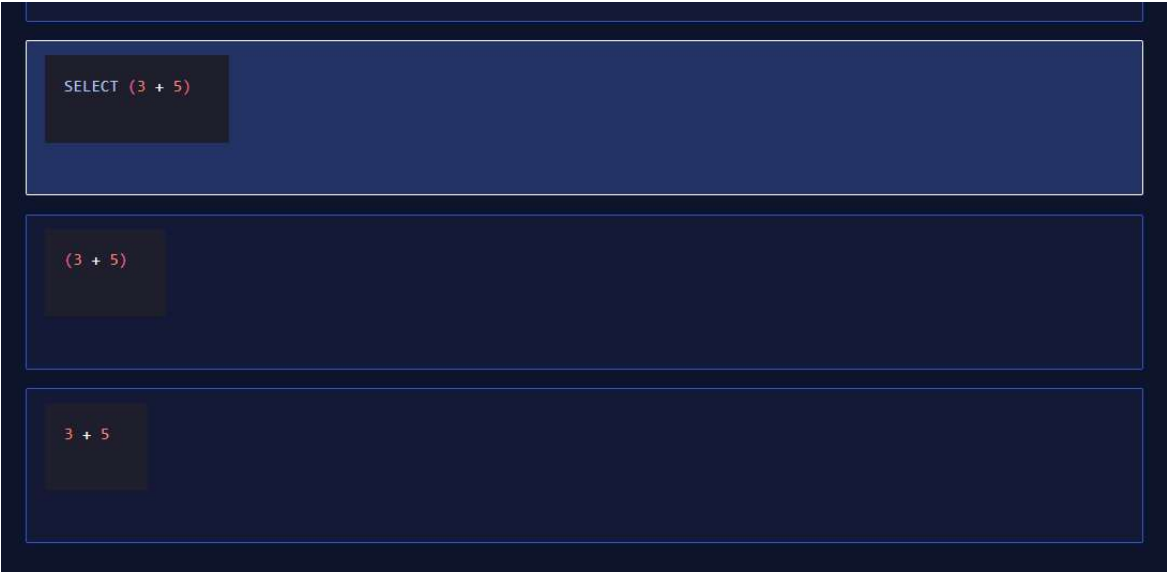
%3

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Within SQL, how would you add the numbers 3 and 5?

SELECT

(3 + 5)



Option	Validity	Result	Why
SELECT (3 + 5)	✔ Correct	8	Proper SQL syntax SELECT and +
SELECT (3 - 5)	✘ Wrong operator	-2	Subtracts instead of adds
(3 + 5)	✘ Missing SELECT	Syntax error	No query keyword
3 + 5	✘ Missing SELECT	Syntax error	No query keyword

Operator	Operation	Example	Result	
+	Addition	SELECT 3 + 5;	8	
-	Subtraction	SELECT 10 - 4;	6	

*	Multiplication	SELECT 6 * 7;	42	
/	Division	SELECT 10 / 2;	5	
%	Modulo (remainder)	SELECT 10 % 3;	1	

Our input date string is 2020-10-15. What modifiers do we need in the function to get the output date string as 2020-10-04?

```
SELECT DATE('2020-10-15', , )
```

'+3 days' '-11 days' '-3 days' '+3 days' 'start of month' 'start of year'

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Input	Target	Calculation
2020-10-15	2020-10-04	Subtract 11 days (15 - 4 =

Modifier	Result from '2020-10-15'	Why Incorrect
'+3 days'	2020-10-18	Moves forward, not backwa
'-3 days'	2020-10-12	Only subtracts 3 days, nee
'start of month'	2020-10-01	Close but needs +3 days m
'start of year'	2020-01-01	Wrong month entirely

Modifier	Effect
'-N days '	Subtract N days
'+N days '	Add N days
'-N months '	Subtract N months
'+N months '	Add N months
'start of month '	First day of current month
'start of year '	First day of current year
'weekday N '	Move to nearest weekday (0=Sunday, 6=Saturday)

Step	Modifier	Resulting date	Explanation	
1	'start of month'	2020-10-01	Moves to the first day of October	
2	'+3 days'	2020-10-04	Adds 3 days → lands on the 4th	

Divide 7 by 2 so that you have decimal points in your answers.

```
SELECT  (7 AS  ) / 2  
-- answer should be 3.5!
```

DATETIME

INT

REAL

DIVIDE

DATATYPE

CAST

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Divide 7 by 2 so that you have decimal points in your answers.

```
SELECT  (7 AS  ) / 2  
-- answer should be 3.5!
```

DATETIME

INT

DIVIDE

DATATYPE

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Query	Result	Why
<code>SELECT 7 / 2;</code>	<code>3</code>	Integer ÷ Integer = Integer truncated)
<code>SELECT CAST(7 AS REAL) / 2;</code>	<code>3.5</code>	Floating-point ÷ Integer = [preserved
<code>SELECT 7.0 / 2;</code>	<code>3.5</code>	Using decimal literal also w

Data Type	Description	Example
<code>INTEGER</code>	Whole number	<code>CAST('42' AS INTEGER)</code>
<code>REAL</code>	Floating-point decimal	<code>CAST('3.14' AS REAL)</code>
<code>TEXT</code>	String/text	<code>CAST(123 AS TEXT)</code> →
<code>NUMERIC</code>	Numeric precision	<code>CAST('42.5' AS NUMERI</code>

Which format string would extract seconds from a time string?

'%H'

'%M'

'%S'

'%seconds'

Which format string would extract seconds from a time string?

'%H'

'%M'

'%S'

'%seconds'

Format Specifier	Meaning	Example Output
------------------	---------	----------------

'%H'	Hours (00–23)	14 from '2021-07-19 :
'%M'	Minutes (00–59)	30 from '2021-07-19 :
'%S'	Seconds (00–59)	45 from '2021-07-19 : 
'%seconds'	Not valid	Syntax error or returns liter

Mistake	Problem	Correct Usage
'%m' vs '%M'	Confusing month (lowercase) with minutes (uppercase)	%m = month, %M = minu
'%s' vs '%S'	Lowercase s = Unix timestamp , uppercase S = seconds	%S for seconds of minute epoch
'%seconds'	Not a valid specifier	Always use single-letter co
Missing quotes	Format strings must be quoted	'%' not %

Specifier	Meaning	Example
'%s' (lowercase)	Unix timestamp (seconds since 1970-01-01)	1626724245
'%S' (uppercase)	Seconds within the minute (00-59)	45

Which `modifier` here shifts the date backwards when inserted into the following function?

```
SELECT DATE('2012-04-10', modifier)
```

`'start of the day'`

`'+3 days'`

`'-3 days'`

`'+20 days'`

Modifier	Effect	Result from '2012-04-10'
'start of the day'	Resets time (no shift)	2012-04-10
'+3 days'	Adds days (forward)	2012-04-13
'-3 days'	Subtracts days (backward)	2012-04-07 
'+20 days'	Adds days (forward)	2012-04-30

Sign	Meaning	Example
-	Shift backward in time	'-1 month'
+	Shift forward in time	'+1 month'
(none)	Neutral operations	'start of month'

Option	Direction	Why Incorrect
'start of the day'	Neutral	Sets time to midnight, does
'+3 days'	Forward	Adds days, not subtracts
'+20 days'	Forward	Adds days, not subtracts

Which `modifier` here shifts the date backwards when inserted into the following function?

```
SELECT DATE('2012-04-10', modifier)
```


'start of the day'

'+3 days'

'-3 days'

'+20 days'

How would you extract the hour from this datetime string: '2015-12-07 17:44:01'?

SELECT (, '2015-12-07 17:44:01')

'%H'

EXTRACT

DATETIME

'%Y'

STRFTIME

'%h'

DATE

Click or drag and drop to fill in the blanks. When finished click Check Answer.

How would you extract the hour from this datetime string: '2015-12-07 17:44:01'?

SELECT (, '2015-12-07 17:44:01')

SELECT

STRFTIME

(

'%H'

,

2015-12-07 17:44:01

)

EXTRACT

DATETIME

'%Y'

'%h'

DATE

Click or drag and drop to fill in the blanks. When finished click Check Answer.

Format Specifier	Meaning	Example Output	
'%H'	Hour (00–23)	'17' from '17:44:01' 	
'%M'	Minutes (00–59)	'44'	
'%S'	Seconds (00–59)	'01'	
'%Y'	Year (4 digits)	'2015'	
'%m'	Month (01–12)	'12'	
'%d'	Day (01–31)	'07'	

Specifier	Meaning	Common Mistake
%H	Hour (00-23)	✓ Correct
%h	Not standard (may vary)	✗ Wrong for hours
%m	Month (01-12)	✓ Lowercase m
%M	Minutes (00-59)	✓ Uppercase M
%Y	Year (4 digits)	✓ Uppercase Y

Option	Problem	Correct Alternative
EXTRACT	PostgreSQL/SQL Server syntax	Use STRFTIME() in SQL
DATETIME	Returns full datetime, not just hour	Use STRFTIME('%H', ..
DATE	Returns date only, no time info	Use TIME() or STRFTIME('%H', ...)
'%h'	Not valid for hours	Use '%H' (uppercase)
'%Y'	Extracts year, not hour	Use '%H'

Today's date is 2021-07-19. You order a custom t-shirt and your stylist assures you that it will be done in 7 days. You end up adding a design which delays the completion by another 3 hours. What is the correct SQL syntax and function that you would use to calculate the date and time of completion?

```
SELECT DATETIME('2021-07-19', '+7 days', '+3 hours')
```

```
SELECT DATETIME('2021-07-19', '+7 days')
```

```
SELECT DATE('2021-07-19', '+7 days', '+3 hours')
```

```
SELECT STRFTIME('%m', '2021-07-19')
```

Today's date is 2021-07-19. You order a custom t-shirt and your stylist assures you that it will be done in 7 days. You end up adding a design which delays the completion by another 3 hours. What is the correct SQL syntax and function that you would use to calculate the date and time of completion?

```
SELECT DATETIME('2021-07-19', '+7 days', '+3 hours')
```

```
SELECT DATETIME('2021-07-19', '+7 days')
```

```
SELECT DATE('2021-07-19', '+7 days', '+3 hours')
```

```
SELECT STRFTIME('%m', '2021-07-19')
```

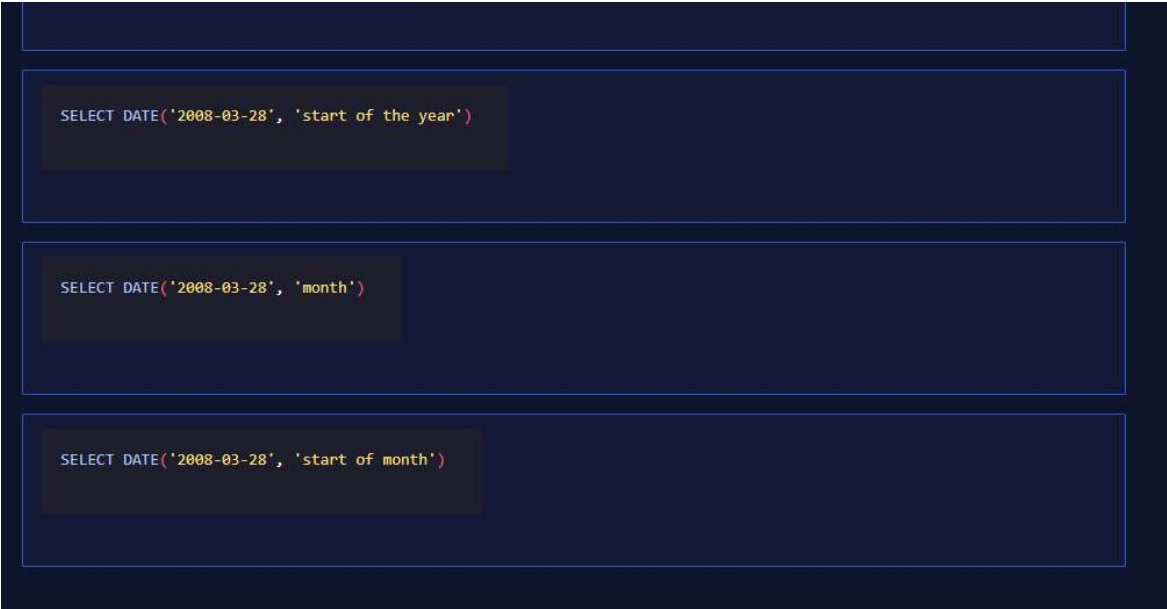
Function	Returns	Example Result
<code>DATETIME()</code>	Date + Time	<code>'2021-07-26 00:00:00'</code>
<code>DATE()</code>	Date only	<code>'2021-07-26'</code> (no time i
<code>TIME()</code>	Time only	<code>'00:00:00'</code> (no date inf
<code>STRFTIME()</code>	Custom formatted string	Depends on format specific

Option	Problem	Result
<code>DATETIME('2021-07-19', '+7 days')</code>	Missing the +3 hours modifier	<code>'2021-07-26 00:00:00'</code>
<code>DATE('2021-07-19', '+7 days', '+3 hours')</code>	<code>DATE()</code> ignores time modifiers	<code>'2021-07-26'</code> (loses 3 h
<code>STRFTIME('%m', '2021-07-19')</code>	Extracts month , doesn't calculate future date	<code>'07'</code> (completely wrong)

Modifier	Effect
' +N days '	Add N days
' +N hours '	Add N hours
' +N minutes '	Add N minutes
' +N seconds '	Add N seconds
' -N days '	Subtract N days
' start of day '	Reset time to midnight
' now '	Current date/time

From the date 2008-03-28, you'd like to get the start of the month, which is 2008-03-01. What is the correct SQL function and syntax?

```
SELECT STFTIME('%m', '2008-03-28')
```

Option	Problem	Result
<code>STRFTIME('%m', '2008-03-28')</code>	Extracts month number only, doesn't return a date	<code>'03'</code>
<code>DATE('2008-03-28', 'start of the year')</code>	Goes to start of the year , not month	<code>'2008-01-01'</code>
<code>DATE('2008-03-28', 'month')</code>	<code>'month'</code> is not a valid modifier	Invalid/Error
<code>DATE('2008-03-28', 'start of month')</code>	✔ Correct! Returns first day of the month	<code>'2008-03-01'</code>

Modifier	Effect	Example (from '2008-03-2
'start of month'	First day of current month	'2008-03-01'
'start of year'	First day of current year	'2008-01-01'
'start of day'	Resets time to midnight	'2008-03-28 00:00:00'

From the date `2008-03-28`, you'd like to get the start of the month, which is `2008-03-01`. What is the correct SQL function and syntax?

```
SELECT STFRIME('%m', '2008-03-28')
```

```
SELECT DATE('2008-03-28', 'start of the year')
```

```
SELECT DATE('2008-03-28', 'month')
```

```
SELECT DATE('2008-03-28', 'start of month')
```

